
accelerate
collaborate
innovate
communicate
palm
PalmSource
Conference 2000



Optimizing Use of the Serial Manager

Bruce Thompson
Software Engineer, Palm, Inc.

PalmSource
Conference 2000

Optimizing Use of the Serial Manager

- Why and how should you optimize?
- Conflicting Goals
  - Raw Throughput
  - I/O Response
  - UI Response
- Tools for optimization
- Don’t optimize too soon!

Goals in Conflict

- These three goals have conflicting requirements
  - Raw Throughput fights UI Response
  - UI Response fights I/O Response
  - I/O Response fights Raw Throughput
- Choose your goal early then optimize for it later

Optimizing for Raw Throughput

- Tailor the buffer size to your apps ability to drain it
- Read the data in big chunks. Keep the input buffer empty
- Respond first to incoming data, then to UI events
- Use the Window calls to read directly from the input buffer

Sample Main Event Loop

- Loop
  - Check for I/O
  - SrmReceiveWindowOpen
  - Pull out all pending Input
  - SrmReceiveWindowClose
  - Handle pending I/O
  - Check for User Event
- End loop

Optimizing for Responsiveness to I/O

- Use a wakeup handler to notify your app of incoming data
- Drop everything when there is data to respond to
- Transfer data out-of-band if need be
- Write a virtual serial driver

Sample Main Event Loop

- Set Wakeup Handler
- Prime Wakeup Handler
- Queue KeyPress (in wakeup handler when triggered)
- Loop
  - Check for event
  - If (key event from handler)
    - SrmReceiveWindowOpen
    - Copy out the I/O
    - SrmReceiveWindowClose
    - Prime the wakeup handler
  - End if
- End Loop

Optimizing for Responsiveness to UI Events

- Don’t ever block!
- Let the input buffer do its job
- Keep all I/O within the event loop and check for UI events often
- Perform as little I/O work as you can afford to between loops

Sample Main Event Loop

- Loop
  - Wait for UI Event (with timeout)
  - Handle UI Event
  - SrmReceiveWindowOpen (quick way to check for and get pending I/O at the same time)
  - Copy one chunk out
  - SrmReceiveWindowClose
- End loop

Tools the API Provides

- SrmSetWakeupHandler
- SrmPrimeWakeupHandler
- SrmReceiveWindowOpen
- SrmReceiveWindowClose

Wakeup Handlers

- Run at Interrupt time
- Fastest way to be notified that input has arrived
- Tight constraints on what a handler can do
- Principle use to notify UI app that Input is pending

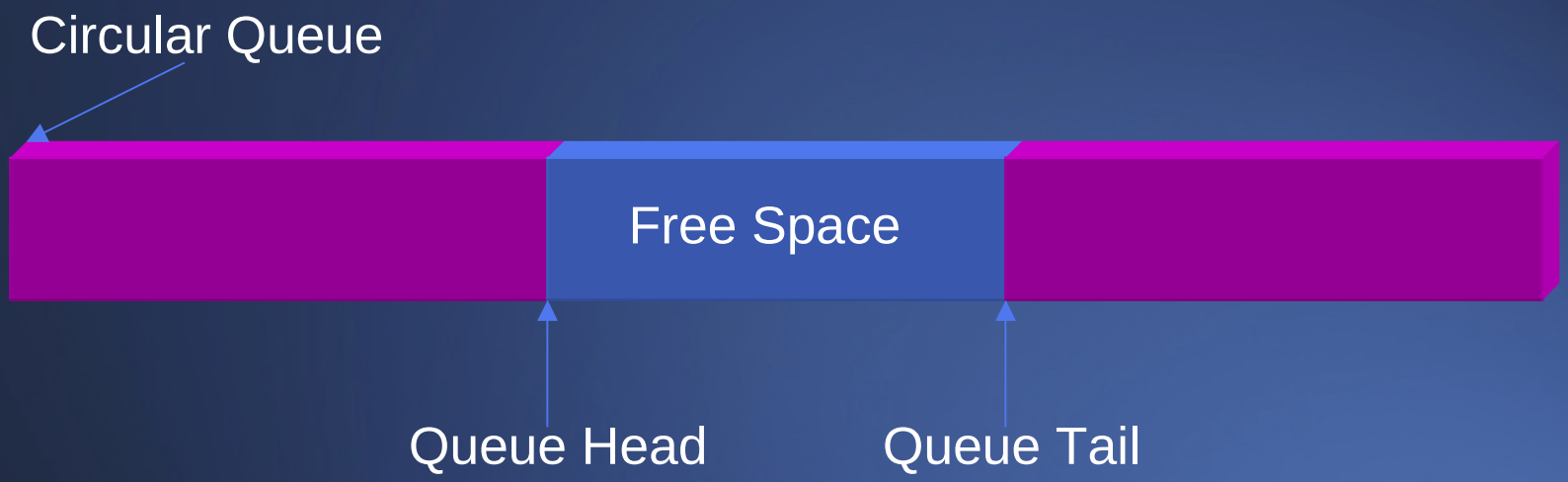
Wakeup Handlers (Cont.)

- Must be primed
- Usually primed for 1 byte
- Typically enqueues a key event to notify the UI app

Receive Windows

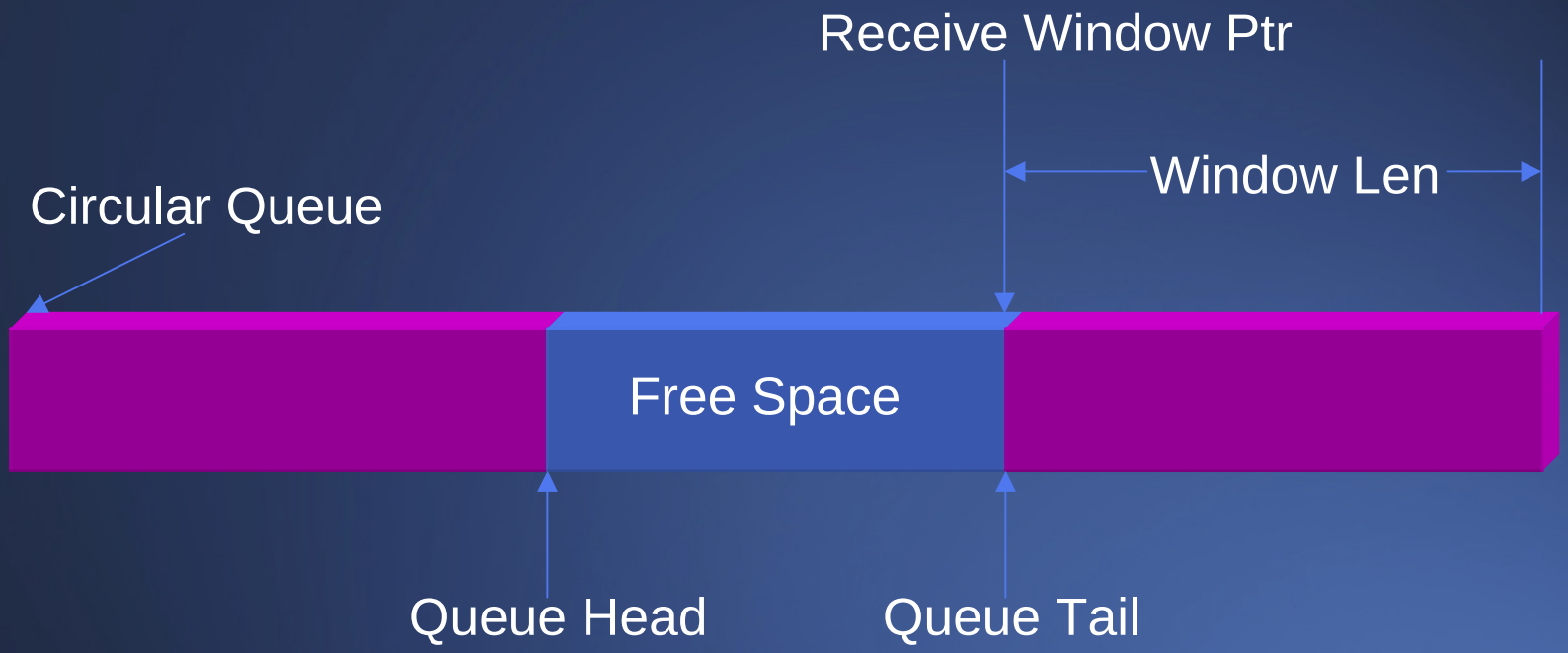
- Fastest way to get data out of the Serial Manager
- Circular queue
- Close it as soon as possible

Receive Windows



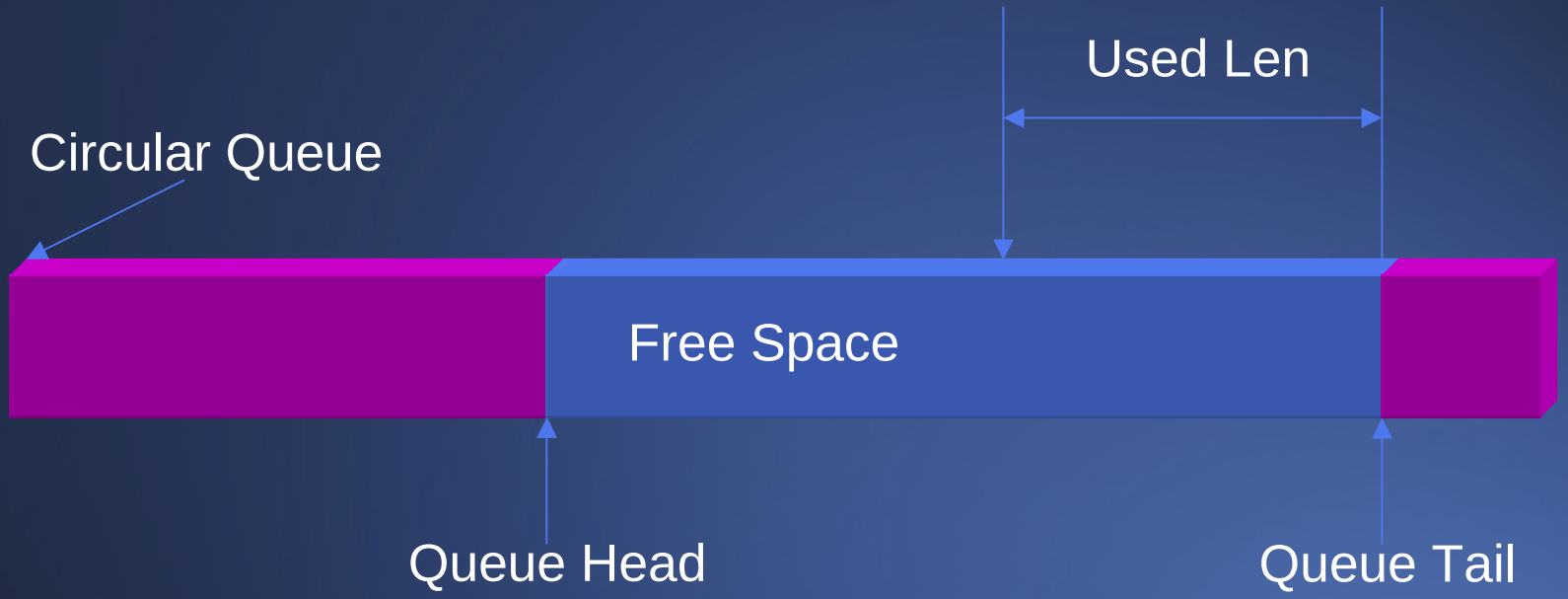
Receive Windows

SrmReceiveWindowOpen



Receive Windows

SrmReceiveWindowClose



Balancing These Requirements

- Often it's not as simple as all this
- Balance the needs of the UI with the needs of the I/O
- Stop when it's good enough!


accelerate
collaborate
innovate
communicate
palm
Q&A


accelerate
collaborate
innovate
communicate
palm
PalmSource
Conference 2000